

数学与思维题算法专题

算法学习笔记

2026 年 5 月 8 日

目录

1	基础数学思维	3
1.1	试除法判断质数	3
1.2	试除法分解质因数	4
1.3	埃拉托色尼筛法	5
1.4	线性筛（欧拉筛）	6
1.5	最大公约数（辗转相除法）	7
1.6	最小公倍数	7
1.7	整数二分查找	8
1.8	快速排序（分治思维）	9
1.9	前缀和	10
1.10	差分数组	11
2	高级数学算法	12
2.1	快速幂	12
2.2	扩展欧几里得算法	13
2.3	乘法逆元	14
2.4	线性递推逆元	15
2.5	组合数（递推法）	16
2.6	组合数（阶乘逆元法）	17
2.7	卢卡斯定理	18
2.8	哥德巴赫猜想验证	19
3	高级专题	20
3.1	高斯消元解线性方程组	20
3.2	高斯消元解异或方程组	22
3.3	矩阵快速幂求斐波那契数列	23

3.4	矩阵快速幂加速一般递推	24
3.5	容斥原理	26
3.6	博弈论——Nim 游戏	27
3.7	博弈论——SG 函数	28
3.8	博弈论——SG 函数（一般情况）	28
3.9	中国剩余定理（CRT）	30
4	算法总结	31

1 基础数学思维

1.1 试除法判断质数

解决问题：快速判断一个正整数是否为质数。适用于加密算法基础、数学验证、质因子分解的预处理步骤。

题目描述

给定一个正整数 n ，判断它是否为质数。若 n 只能被 1 和自身整除，则为质数。

输入示例

17

输出示例

YES

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 待判断的整数
5          cin >> n;
6          if (n <= 1) { // 1及以下的数不是质数
7              cout << "NO\n";
8              return 0;
9          }
10         bool isPrime = true; // isPrime: 标记是否为质数，初始假设为真
11         // 核心原理：从2遍历到sqrt(n)，若存在整除则说明存在非1和本身的因子
12         for (int i = 2; i * i <= n; i++) { // i: 尝试的因子
13             if (n % i == 0) { // 找到因子，说明不是质数
14                 isPrime = false;
15                 break;
16             }
17         }
18         cout << (isPrime ? "YES" : "NO") << "\n";
19         return 0;
20     }
```

1.2 试除法分解质因数

解决问题：将一个正整数分解为若干个质数相乘的形式。适用于数论基础、最大公约数/最小公倍数求解、密码学中的 RSA 原理解释。

题目描述

给定一个正整数 n ，将其分解成质因数的乘积形式。例如 $12 = 2^2 \times 3$ 。

输入示例

60

输出示例

$2^2 * 3^1 * 5^1$

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 待分解的正整数
5          cin >> n;
6          int temp = n; // temp: n的副本，用于循环处理
7          bool first = true; // first: 控制输出格式，是否输出乘号
8          // 核心原理：从2开始循环，找到因子就不断除以该因子并计数
9          for (int i = 2; i * i <= temp; i++) { // i: 尝试的质因子
10             if (temp % i == 0) {
11                 int cnt = 0; // cnt: 当前质因子的指数
12                 while (temp % i == 0) { // 只要还能整除，就继续除
13                     temp /= i;
14                     cnt++;
15                 }
16                 if (!first) cout << " * ";
17                 cout << i << "^" << cnt;
18                 first = false;
19             }
20         }
21         if (temp > 1) { // 剩余一个大质数因子
22             if (!first) cout << " * ";
23             cout << temp << "^1";
24         }
```

```
25     cout << "\n";
26     return 0;
27 }
```

1.3 埃拉托色尼筛法

解决问题：高效求出 1 到 n 范围内的所有质数。时间复杂度 $O(n \log \log n)$ 。适用于数据范围较大的质数表生成、质数相关问题预处理。

题目描述

给定一个整数 n ，输出 1 到 n 之间的所有质数。

输入示例

20

输出示例

2 3 5 7 11 13 17 19

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n; // n: 筛选范围的上界
5      cin >> n;
6      vector<bool> isPrime(n + 1, true); // isPrime[i]: 标记i是否为质数，初
7      始全为true
8      isPrime[0] = isPrime[1] = false; // 0和1不是质数
9      // 核心原理：从小到大，若当前数为质数，则筛掉其所有倍数
10     for (int i = 2; i * i <= n; i++) { // i: 当前用于筛除倍数的质数
11         if (isPrime[i]) {
12             for (int j = i * i; j <= n; j += i) { // j: i的倍数，从i*i开始
13                 更高效
14                 isPrime[j] = false; // 标记i的倍数为合数
15             }
16         }
17     }
18     for (int i = 2; i <= n; i++) {
19         if (isPrime[i]) cout << i << " ";
20     }
```

```
19     cout << "\n";
20     return 0;
21 }
```

1.4 线性筛（欧拉筛）

解决问题：在线性时间复杂度 $O(n)$ 内求出 1 到 n 的所有质数，并能计算每个数的最小质因子。适用于需要高效筛法的大数据范围问题。

题目描述

给一个整数 n ，列出不大于 n 的所有质数。

输入示例

20

输出示例

2 3 5 7 11 13 17 19

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n; // n: 筛选范围的上界
5      cin >> n;
6      vector<int> primes; // primes: 存储找到的质数
7      vector<bool> isComp(n + 1, false); // isComp[i]: 标记i是否为合数
8      // 核心原理：每个合数只被其最小质因子筛掉一次，保证线性复杂度
9      for (int i = 2; i <= n; i++) { // i: 当前处理的数
10         if (!isComp[i]) primes.push_back(i); // i是质数则加入列表
11         for (int p : primes) { // p: 质数列表中的质数
12             if (i * p > n) break; // 超过范围则停止
13             isComp[i * p] = true; // 筛掉合数
14             if (i % p == 0) break; // p是i的最小质因子，停止避免重复筛
15         }
16     }
17     for (int p : primes) cout << p << " ";
18     cout << "\n";
19     return 0;
20 }
```

1.5 最大公约数（辗转相除法）

解决问题：求两个整数的最大公约数（GCD）。适用于分数约分、最小公倍数计算、数论问题的基础工具。

题目描述

给定两个非负整数 a 和 b ，求它们的最大公约数。

输入示例

48 18

输出示例

6

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      // 递归实现辗转相除法，核心原理：gcd(a, b) = gcd(b, a % b)，直到b为0返回a
4      int gcd(int a, int b) { // a, b: 两个非负整数
5          return b == 0 ? a : gcd(b, a % b); // 递归直到b为0
6      }
7      int main() {
8          int a, b; // a, b: 输入的两个整数
9          cin >> a >> b;
10         cout << gcd(a, b) << "\n";
11         return 0;
12     }
```

1.6 最小公倍数

解决问题：求两个整数的最小公倍数（LCM），通过公式 $lcm(a, b) = a / gcd(a, b) \times b$ 计算。适用于数学计算、周期性事件同步问题。

题目描述

给定两个正整数 a 和 b ，求它们的最小公倍数。

输入示例

12 18

输出示例

36

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
4      int lcm(int a, int b) { // a, b: 两个正整数
5          return a / gcd(a, b) * b; // 先除后乘防止溢出
6      }
7      int main() {
8          int a, b; // a, b: 输入的两个正整数
9          cin >> a >> b;
10         cout << lcm(a, b) << "\n";
11         return 0;
12     }
```

1.7 整数二分查找

解决问题：在有序序列中快速查找目标值，或查找满足单调条件的边界。时间复杂度 $O(\log n)$ 。适用于有序数组搜索、最大值最小化/最小值最大化等答案二分问题。

题目描述

给定一个升序数组和一个目标值 x ，找出 x 第一次出现的位置，若不存在输出 -1 。

输入示例

```
5 3
1 2 3 3 5
```

输出示例

2（下标从 0 开始）

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, x; // n: 数组长度, x: 目标值
5          cin >> n >> x;
6          vector<int> a(n); // a: 升序数组
7          for (int i = 0; i < n; i++) cin >> a[i];
```



```
8      int l = 0, r = n - 1, ans = -1; // l: 左边界, r: 右边界, ans: 记录答
      案位置
9      // 核心原理: 在有序区间内不断取中点, 根据中点值与目标比较缩小搜索范围
10     while (l <= r) {
11         int mid = l + (r - l) / 2; // mid: 当前搜索区间的中点
12         if (a[mid] >= x) { // 中点值大于等于目标, 答案可能在左半部分
13             ans = mid;
14             r = mid - 1;
15         } else { // 中点值小于目标, 答案在右半部分
16             l = mid + 1;
17         }
18     }
19     if (ans != -1 && a[ans] == x) cout << ans << "\n";
20     else cout << -1 << "\n";
21     return 0;
22 }
```

1.8 快速排序（分治思维）

解决问题：对一个数组进行快速排序，平均时间复杂度 $O(n \log n)$ 。分治思想是许多高效算法的基础，如快速选择、归并排序求逆序对等。

题目描述

读入 n 个整数，将它们从小到大排序后输出。

输入示例

```
5
3 1 4 1 5
```

输出示例

```
1 1 3 4 5
```

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      // 核心原理: 选择一个基准值, 将数组分为小于和大于基准的两部分, 递归处理
4      void quickSort(vector<int>& a, int l, int r) { // a: 待排序数组, l: 左边界, r: 右边界
5          if (l >= r) return;
```

```
6     int pivot = a[l + (r - l) / 2]; // pivot: 基准值, 取中间位置的值
7     int i = l, j = r; // i: 左指针, j: 右指针
8     while (i <= j) {
9         while (a[i] < pivot) i++; // 找到左边不小于基准的元素
10        while (a[j] > pivot) j--; // 找到右边不大于基准的元素
11        if (i <= j) {
12            swap(a[i], a[j]); // 交换后两指针向中间移动
13            i++; j--;
14        }
15    }
16    quickSort(a, l, j); // 递归处理左半部分
17    quickSort(a, i, r); // 递归处理右半部分
18 }
19 int main() {
20     int n; // n: 数组长度
21     cin >> n;
22     vector<int> a(n); // a: 存储数组元素
23     for (int i = 0; i < n; i++) cin >> a[i];
24     quickSort(a, 0, n - 1);
25     for (int x : a) cout << x << " ";
26     cout << "\n";
27     return 0;
28 }
```

1.9 前缀和

解决问题：多次查询一个数组的区间和，预处理 $O(n)$ ，每次查询 $O(1)$ 。适用于频繁区间求和、二维子矩阵求和等问题。

题目描述

给定一个长度为 n 的数组，有 q 个查询，每个查询给出区间 $[l, r]$ ，求该区间元素和。

输入示例

```
5 3
1 2 3 4 5
1 3
2 5
```

0 4

输出示例

6 14 15

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, q; // n: 数组长度, q: 查询次数
5          cin >> n >> q;
6          vector<int> a(n); // a: 存储原始数组
7          for (int i = 0; i < n; i++) cin >> a[i];
8          vector<long long> pre(n + 1, 0); // pre[i]: 前i个元素的前缀和, pre
           [0]=0
9          for (int i = 1; i <= n; i++) {
10             pre[i] = pre[i - 1] + a[i - 1]; // 计算前缀和
11         }
12         while (q--) {
13             int l, r; // l: 区间左端点, r: 区间右端点
14             cin >> l >> r;
15             // 区间和 = 前缀和[r+1] - 前缀和[l]
16             cout << pre[r + 1] - pre[l] << "\n";
17         }
18         return 0;
19     }

```

1.10 差分数组

解决问题: 多次对数组的某个区间进行统一的增减操作, 最后输出结果数组。修改 $O(1)$, 最终还原 $O(n)$ 。适用于区间修改、航班预订、温度变化统计等场景。

题目描述

有一个初始全为 0 的长度为 n 的数组, 进行 m 次操作, 每次对区间 $[l, r]$ 增加 c , 求最终数组。

输入示例

5 3

```

0 2 1
1 4 2
2 3 3

```

输出示例

```
1 3 5 2 0
```

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n, m; // n: 数组长度, m: 操作次数
5          cin >> n >> m;
6          vector<int> diff(n + 2, 0); // diff[i]: 差分数组, 范围开n+2防止越界
7          // 核心原理: 区间[l, r]加c, 等价于diff[l]+=c, diff[r+1]-=c
8          while (m--) {
9              int l, r, c; // l: 区间左端, r: 区间右端, c: 增加的值
10             cin >> l >> r >> c;
11             diff[l] += c; // 左端点加c
12             diff[r + 1] -= c; // 右端点后一个位置减c
13         }
14         vector<int> ans(n); // ans: 最终结果数组
15         int cur = 0; // cur: 当前累积的差分值
16         for (int i = 0; i < n; i++) {
17             cur += diff[i]; // 累加得到当前元素的值
18             ans[i] = cur;
19         }
20         for (int x : ans) cout << x << " ";
21         cout << "\n";
22         return 0;
23     }

```

2 高级数学算法

2.1 快速幂

解决问题：高效计算 $a^b \bmod m$ 的值，时间复杂度 $O(\log b)$ 。适用于大指数取模运算、RSA 加密解密、组合数取模中求逆元等场景。

题目描述

给定三个整数 a, b, m ，求 $a^b \bmod m$ 。

输入示例

2 10 1000

输出示例

24

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      // 核心原理：将指数b转为二进制，利用  $a^{(2^k)} = (a^{(2^{k-1})})^2$  迭代计算
5      ll quickPow(ll a, ll b, ll m) { // a: 底数, b: 指数, m: 模数
6          ll res = 1; // res: 累乘结果
7          a %= m; // 先取模防止溢出
8          while (b > 0) {
9              if (b & 1) res = (res * a) % m; // 二进制当前位为1时，乘上对应的权重
10             a = (a * a) % m; // 底数平方，对应下一位的权重
11             b >>= 1; // 右移处理下一位
12         }
13         return res;
14     }
15     int main() {
16         ll a, b, m; // a: 底数, b: 指数, m: 模数
17         cin >> a >> b >> m;
18         cout << quickPow(a, b, m) << "\n";
19         return 0;
20     }
```

2.2 扩展欧几里得算法

解决问题：求解方程 $ax + by = \gcd(a, b)$ 的一组整数解，以及求逆元、求解线性同余方程。适用于密码学逆元计算、不定方程通解等场景。

题目描述

给定整数 a, b , 求 x, y 使得 $ax + by = \gcd(a, b)$ 。

输入示例

35 15

输出示例

gcd=5, x=1, y=-2

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      // 核心原理：在辗转相除法回溯时，倒推出x和y的递推关系
4      int exgcd(int a, int b, int& x, int& y) { // a,b: 系数, x,y: 解（引用返回）
5          if (b == 0) { // 递归基：gcd(a,0)=a, 此时ax+0*y=a -> x=1,y=0
6              x = 1; y = 0;
7              return a;
8          }
9          int x1, y1; // x1, y1: 下层递归的解
10         int d = exgcd(b, a % b, x1, y1); // d: 最终的gcd
11         x = y1; // x = 下层y1
12         y = x1 - (a / b) * y1; // y = 下层x1 - (a/b)*下层y1
13         return d;
14     }
15     int main() {
16         int a, b; // a, b: 输入的两个整数
17         cin >> a >> b;
18         int x, y; // x, y: 方程的解
19         int d = exgcd(a, b, x, y);
20         cout << "gcd=" << d << ", x=" << x << ", y=" << y << "\n";
21         return 0;
22     }

```

2.3 乘法逆元

解决问题：求整数 a 在模 m 下的乘法逆元，即满足 $a \times x \equiv 1 \pmod{m}$ 的 x 。适用于组合数取模、分数取模等场景。

题目描述

给定 a 和质数 p ，求 a 模 p 的乘法逆元。保证 $\gcd(a, p) = 1$ 。

输入示例

3 7

输出示例

5

输出解释

$$3 \times 5 = 15 \equiv 1 \pmod{7}$$

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      ll quickPow(ll a, ll b, ll m) {
5          ll res = 1; a %= m;
6          while (b) { if (b & 1) res = res * a % m; a = a * a % m; b >>= 1; }
7          return res;
8      }
9      int main() {
10         ll a, p; // a: 需求逆元的数, p: 模数 (质数)
11         cin >> a >> p;
12         // 费马小定理:  $a^{p-1} \equiv 1 \pmod{p}$ , 因此逆元为  $a^{p-2} \pmod{p}$ 
13         cout << quickPow(a, p - 2, p) << "\n";
14         return 0;
15     }

```

2.4 线性递推逆元

解决问题：在 $O(n)$ 时间内预处理 1 到 n 所有数在模质数 p 下的逆元。适用于组合数计算中大量使用阶乘逆元的场景。

题目描述

给定 n 和质数 p ，计算 1 到 n 每个整数模 p 的乘法逆元。

输入示例

5 7

输出示例

1 4 5 2 3

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      int main() {
5          int n; ll p; // n: 上限, p: 模数 (质数)
6          cin >> n >> p;
7          vector<ll> inv(n + 1); // inv[i]: i的逆元
8          inv[1] = 1; // 1的逆元为1
9          // 核心原理: 利用递推公式 inv[i] = p - p/i * inv[p%i] % p
10         for (int i = 2; i <= n; i++) {
11             inv[i] = p - (p / i) * inv[p % i] % p;
12         }
13         for (int i = 1; i <= n; i++) cout << inv[i] << " ";
14         cout << "\n";
15         return 0;
16     }

```

2.5 组合数（递推法）

解决问题：在 $O(n^2)$ 预处理后， $O(1)$ 查询组合数 $C(n, k) \bmod p$ ，适用于 n 较小（ $n \leq 5000$ ）的组合数频繁查询场景。

题目描述

给定 n 和 k ，求组合数 $C(n, k) \bmod (10^9 + 7)$ 。保证 $0 \leq k \leq n \leq 2000$ 。

输入示例

5 2

输出示例

10


```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const int MOD = 1e9 + 7;
4      int main() {
5          int n, k; // n: 总数, k: 选取数
6          cin >> n >> k;
7          vector<vector<int>> C(n + 1, vector<int>(n + 1, 0)); // C[i][j]: C(i
            , j)
8          // 核心原理: 利用递推公式 C(i, j) = C(i-1, j-1) + C(i-1, j)
9          for (int i = 0; i <= n; i++) {
10             C[i][0] = C[i][i] = 1; // 边界条件
11             for (int j = 1; j < i; j++) {
12                 C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % MOD;
13             }
14         }
15         cout << C[n][k] << "\n";
16         return 0;
17     }

```

2.6 组合数（阶乘逆元法）

解决问题：在 $O(n)$ 预处理阶乘和阶乘逆元后， $O(1)$ 查询组合数 $C(n, k) \bmod p$ ，适用于 n 较大（ $n \leq 10^6$ ）的场景。

题目描述

给定 n 和 k ，求 $C(n, k) \bmod (10^9 + 7)$ 。

输入示例

10 4

输出示例

210

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      const int MOD = 1e9 + 7;

```

```

5      const int MAXN = 1e6 + 5;
6      ll fact[MAXN], invFact[MAXN]; // fact[i]: i的阶乘, invFact[i]: i阶乘的逆元
7      ll quickPow(ll a, ll b) { ll r = 1; a %= MOD; while(b){ if(b&1) r=r*a%MOD; a=a*a%MOD; b>>=1; } return r; }
8      void precompute() {
9          fact[0] = 1;
10         for (int i = 1; i < MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
11         invFact[MAXN - 1] = quickPow(fact[MAXN - 1], MOD - 2);
12         for (int i = MAXN - 2; i >= 0; i--) invFact[i] = invFact[i + 1] * (i + 1) % MOD;
13     }
14     ll C(int n, int k) { // C(n, k) = n! / (k! * (n-k)!)
15         if (k < 0 || k > n) return 0;
16         return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
17     }
18     int main() {
19         precompute();
20         int n, k; // n: 总数, k: 选取数
21         cin >> n >> k;
22         cout << C(n, k) << "\n";
23         return 0;
24     }

```

2.7 卢卡斯定理

解决问题：当模数 p 为质数且较小，而 n, k 非常大时，使用卢卡斯定理将组合数分解为 p 进制位的组合数乘积。适用于大组合数对小质数取模。

题目描述

求 $C(n, k) \bmod 10007$ ，其中 $n, k \leq 10^{18}$ 。

输入示例

100 50

输出示例

252

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      const int MOD = 10007;
5      ll fact[MOD + 1];
6      ll quickPow(ll a, ll b) { ll r = 1; a %= MOD; while(b){ if(b&1) r=r*a%
          MOD; a=a*a%MOD; b>>=1; } return r; }
7      ll C_small(ll n, ll k) { // 小范围组合数 n,k < MOD
8          if (k > n) return 0;
9          return fact[n] * quickPow(fact[k], MOD - 2) % MOD * quickPow(fact[n
          - k], MOD - 2) % MOD;
10     }
11     ll lucas(ll n, ll k) { // 卢卡斯定理递归分解
12         if (k == 0) return 1;
13         return C_small(n % MOD, k % MOD) * lucas(n / MOD, k / MOD) % MOD;
14     }
15     int main() {
16         fact[0] = 1;
17         for (int i = 1; i < MOD; i++) fact[i] = fact[i - 1] * i % MOD;
18         ll n, k; // n: 总数, k: 选取数
19         cin >> n >> k;
20         cout << lucas(n, k) << "\n";
21         return 0;
22     }

```

2.8 哥德巴赫猜想验证

解决问题：验证一个大于 2 的偶数可以写成两个质数之和（哥德巴赫猜想）。适用于质数性质探索、构造性问题。

题目描述

给定一个偶数 n ($4 \leq n \leq 10^5$)，找到两个质数使其和等于 n 。

输入示例

20

输出示例

3 17

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      int main() {
4          int n; // n: 输入的偶数
5          cin >> n;
6          vector<bool> isPrime(n + 1, true);
7          isPrime[0] = isPrime[1] = false;
8          for (int i = 2; i * i <= n; i++)
9              if (isPrime[i])
10                 for (int j = i * i; j <= n; j += i) isPrime[j] = false;
11             // 暴力枚举第一个质数, 判断n-p是否为质数
12             for (int p = 2; p <= n; p++) { // p: 第一个质数
13                 if (isPrime[p] && isPrime[n - p]) {
14                     cout << p << " " << n - p << "\n";
15                     break;
16                 }
17             }
18             return 0;
19     }

```

3 高级专题

3.1 高斯消元解线性方程组

解决问题：求解 n 元线性方程组，通过初等行变换将增广矩阵化为行阶梯形。适用于电路分析、线性回归、数学建模等场景。

题目描述

给定 n 个 n 元一次方程，求解方程组的解（保证唯一解）。

输入示例

```

2
1 2 5
3 4 11

```

输出示例

1.00 2.00

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      const double EPS = 1e-8;
4      int main() {
5          int n; // n: 方程数和未知数个数
6          cin >> n;
7          vector<vector<double>> a(n, vector<double>(n + 1)); // a: 增广矩阵
8          for (int i = 0; i < n; i++)
9              for (int j = 0; j <= n; j++)
10                 cin >> a[i][j];
11         // 核心原理: 逐列消元, 每次找主元并消去其他行该列的系数
12         for (int i = 0; i < n; i++) { // i: 当前处理列/主元行
13             int pivot = i; // pivot: 主元所在行
14             for (int j = i + 1; j < n; j++) // 选绝对值最大的行作为主元, 提高
15                 // 数值稳定性
16                 if (fabs(a[j][i]) > fabs(a[pivot][i])) pivot = j;
17             swap(a[i], a[pivot]);
18             if (fabs(a[i][i]) < EPS) continue; // 主元为0, 方程组可能无解或无
19                 // 穷解
20             for (int j = i + 1; j <= n; j++) a[i][j] /= a[i][i]; // 归一化主
21                 // 元行
22             a[i][i] = 1;
23             for (int j = 0; j < n; j++) { // j: 消去其他行第i列
24                 if (i == j) continue;
25                 double factor = a[j][i]; // factor: 消元系数
26                 for (int k = i; k <= n; k++) a[j][k] -= factor * a[i][k];
27             }
28         }
29         for (int i = 0; i < n; i++)
30             cout << fixed << setprecision(2) << a[i][n] << " ";
31         cout << "\n";
32         return 0;
33     }

```

3.2 高斯消元解异或方程组

解决问题：求解系数和结果都是 0/1 的异或线性方程组。适用于博弈论、开关灯问题、图论中求割集等场景。

题目描述

有 n 个开关和 n 个灯，每个开关控制一些灯，求一种按开关方案使所有灯亮。

输入示例

```
3
1 1 0 1
0 1 1 0
1 0 1 1
```

输出示例

```
1 1 0
```

```
1      #include <bits/stdc++.h>
2
3      using namespace std;
4      int main() {
5          int n; // n: 方程组阶数
6          cin >> n;
7          vector<bitset<105>> a(n); // a: 增广矩阵, 每个bitset表示一行
8          for (int i = 0; i < n; i++) {
9              for (int j = 0; j <= n; j++) {
10                 int x; cin >> x;
11                 if (x) a[i].set(j); // 第j位设为1
12             }
13         }
14         vector<int> ans(n, 0); // ans: 解向量
15         // 核心原理: 异或方程组的高斯消元, 利用异或操作进行行变换
16         for (int i = 0; i < n; i++) { // i: 当前消元列
17             int pivot = i;
18             while (pivot < n && !a[pivot][i]) pivot++;
19             if (pivot == n) continue; // 该列无主元
20             swap(a[i], a[pivot]);
21             for (int j = 0; j < n; j++) {
22                 if (i != j && a[j][i]) a[j] ^= a[i]; // 异或消元
```

```

22     }
23 }
24 for (int i = 0; i < n; i++) ans[i] = a[i][n];
25 for (int x : ans) cout << x << " ";
26 cout << "\n";
27 return 0;
28 }

```

3.3 矩阵快速幂求斐波那契数列

解决问题：在 $O(\log n)$ 时间内计算斐波那契数列的第 n 项。适用于大范围数列加速计算、线性递推式加速、图论中路径计数等。

题目描述

给定 n ，求斐波那契数列第 n 项模 $10^9 + 7$ 的值。 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ 。

输入示例

10

输出示例

55

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  const int MOD = 1e9 + 7;
5  struct Matrix {
6      ll a[2][2]; // a: 2x2矩阵元素
7      Matrix() { memset(a, 0, sizeof(a)); }
8      Matrix operator*(const Matrix& o) const { // 矩阵乘法
9          Matrix res;
10         for (int i = 0; i < 2; i++)
11             for (int j = 0; j < 2; j++)
12                 for (int k = 0; k < 2; k++)
13                     res.a[i][j] = (res.a[i][j] + a[i][k] * o.a[k][j]) % MOD;
14         return res;
15     }
16 };

```

```

17     Matrix matPow(Matrix base, ll exp) { // 矩阵快速幂
18         Matrix res; res.a[0][0] = res.a[1][1] = 1; // 单位矩阵
19         while (exp) {
20             if (exp & 1) res = res * base;
21             base = base * base;
22             exp >>= 1;
23         }
24         return res;
25     }
26     int main() {
27         ll n; // n: 求第n项
28         cin >> n;
29         if (n == 0) { cout << 0 << "\n"; return 0; }
30         Matrix base; // base: 转移矩阵 [1 1; 1 0]
31         base.a[0][0] = 1; base.a[0][1] = 1;
32         base.a[1][0] = 1; base.a[1][1] = 0;
33         Matrix res = matPow(base, n - 1);
34         cout << res.a[0][0] << "\n"; // 结果就是F(n)
35         return 0;
36     }

```

3.4 矩阵快速幂加速一般递推

解决问题：对于任意线性递推式，构造转移矩阵后用矩阵快速幂加速求解第 n 项。适用于动态规划优化、状态转移计数等问题。

题目描述

已知 a_1, a_2 ，递推 $a_i = p \cdot a_{i-1} + q \cdot a_{i-2}$ ，求第 n 项模 M 。

输入示例

1 2 3 4 5

a1=1 a2=2 p=3 q=4 n=5

输出示例

83


```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      ll MOD;
5      struct Matrix {
6          int sz; vector<vector<ll>> a; // sz: 矩阵大小, a: 矩阵元素
7          Matrix(int s) : sz(s), a(s, vector<ll>(s, 0)) {}
8          Matrix operator*(const Matrix& o) const {
9              Matrix res(sz);
10             for (int i = 0; i < sz; i++)
11                 for (int j = 0; j < sz; j++)
12                     for (int k = 0; k < sz; k++)
13                         res.a[i][j] = (res.a[i][j] + a[i][k] * o.a[k][j]) % MOD;
14             return res;
15         }
16     };
17     Matrix matPow(Matrix base, ll exp) {
18         Matrix res(base.sz);
19         for (int i = 0; i < base.sz; i++) res.a[i][i] = 1;
20         while (exp) { if (exp & 1) res = res * base; base = base * base; exp
21             >>= 1; }
22         return res;
23     }
24     int main() {
25         ll a1, a2, p, q, n; // a1,a2: 初始值, p,q: 系数, n: 目标项数
26         cin >> a1 >> a2 >> p >> q >> n >> MOD;
27         if (n == 1) { cout << a1 % MOD << "\n"; return 0; }
28         if (n == 2) { cout << a2 % MOD << "\n"; return 0; }
29         Matrix base(2); // base: 转移矩阵 [[p, q], [1, 0]]
30         base.a[0][0] = p % MOD; base.a[0][1] = q % MOD;
31         base.a[1][0] = 1; base.a[1][1] = 0;
32         Matrix res = matPow(base, n - 2);
33         ll ans = (res.a[0][0] * a2 % MOD + res.a[0][1] * a1 % MOD) % MOD;
34         cout << ans << "\n";
35         return 0;
36     }

```

3.5 容斥原理

解决问题：计算多个集合并集的元素个数，公式为 $|\bigcup A_i| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 。适用于计数问题、整除统计、错排问题等场景。

题目描述

求 1 到 n 中能被 2 或 3 或 5 整除的数的个数。

输入示例

100

输出示例

74

```

1      #include <bits/stdc++.h>
2      using namespace std;
3      using ll = long long;
4      int main() {
5          ll n; // n: 数字范围上限
6          cin >> n;
7          vector<int> primes = {2, 3, 5}; // primes: 备选质数集
8          int m = primes.size();
9          ll ans = 0; // ans: 符合至少一个条件的数量
10         // 核心原理：枚举所有非空子集，奇数大小子集加，偶数大小子集减
11         for (int mask = 1; mask < (1 << m); mask++) { // mask: 子集的二进制表示
12             ll lcm = 1; // lcm: 当前子集元素的最小公倍数
13             int bits = 0; // bits: 子集大小
14             for (int i = 0; i < m; i++) {
15                 if (mask & (1 << i)) {
16                     bits++;
17                     lcm = lcm * primes[i] / __gcd(lcm, (ll)primes[i]);
18                     if (lcm > n) break;
19                 }
20             }
21             if (lcm > n) continue;
22             ll cnt = n / lcm; // cnt: 当前子集整除数的个数
23             if (bits % 2 == 1) ans += cnt; // 奇数子集加

```

```

24         else ans -= cnt; // 偶数子集减
25     }
26     cout << ans << "\n";
27     return 0;
28 }

```

3.6 博弈论——Nim 游戏

解决问题：判断在 Nim 游戏中先手是否必胜。Nim 游戏：有若干堆石子，两人轮流从任意一堆中取任意数量（至少 1 个），无法操作者输。适用于基本的公平组合游戏胜负判断。

题目描述

有 n 堆石子，第 i 堆有 a_i 个石子。两人轮流操作，每次从一堆中取至少一个石子，判先手胜负。

输入示例

```

3
2 3 4

```

输出示例

First

```

1     #include <bits/stdc++.h>
2     using namespace std;
3     int main() {
4         int n; // n: 堆数
5         cin >> n;
6         int xsum = 0; // xsum: 所有堆石子数的异或和
7         // 核心原理: Nim游戏的Bouton定理: 异或和非零则先手必胜
8         for (int i = 0; i < n; i++) {
9             int a; cin >> a; // a: 第i堆的石子数
10            xsum ^= a;
11        }
12        cout << (xsum ? "First" : "Second") << "\n";
13        return 0;
14    }

```

3.7 博弈论——SG 函数

解决问题：对任意有向图游戏，计算其 SG 函数值，从而判断多游戏组合的胜负。适用于各种组合博弈问题（如取石子变体、棋盘游戏等）。

题目描述

有 n 堆石子，每堆给定数量 a_i ，每次操作可以选择一堆，取走 $1 \sim k$ 个石子。判断先手胜负。

输入示例

```
3 3
2 4 7
```

输出示例

Second

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int n, k; // n: 堆数, k: 每次最多取的石子数
5      cin >> n >> k;
6      // 核心原理: 对于限定取1~k个的巴什博弈, SG(x) = x % (k+1)
7      int xsum = 0; // xsum: 各堆SG值的异或和
8      for (int i = 0; i < n; i++) {
9          int a; cin >> a; // a: 第i堆的石子数
10         xsum ^= (a % (k + 1)); // SG值 = a % (k+1)
11     }
12     cout << (xsum ? "First" : "Second") << "\n";
13     return 0;
14 }
```

3.8 博弈论——SG 函数（一般情况）

解决问题：对于复杂的有向图组合游戏，通过记忆化搜索计算每个局面的 SG 值，然后通过 SG 定理（异或和为 0 则必败）判断组合游戏胜负。

题目描述

有一堆石子，每次可以取集合 S 中任一数量的石子。多组测试，判断单堆石子的胜负。

输入示例

```
2 2 5
3
2 5 12
```

解释： $S=\{2,5\}$ ，查询3堆：2, 5, 12

输出示例

Lose Win Win

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int m; // m: 可取石子数的个数
5      cin >> m;
6      vector<int> S(m); // S: 可取的合法集合
7      for (int i = 0; i < m; i++) cin >> S[i];
8      int maxH = 10000; // maxH: 最大石子数范围
9      vector<int> sg(maxH + 1, -1); // sg[x]: 石子数为x时的SG值
10     // 核心原理: SG(x) = mex{SG(x - s) | s ∈ S且s ≤ x}
11     function<int(int)> calcSG = [&](int x) {
12         if (sg[x] != -1) return sg[x];
13         set<int> mexSet; // mexSet: 记录可达状态的SG值集合
14         for (int s : S) {
15             if (x >= s) mexSet.insert(calcSG(x - s));
16         }
17         int mex = 0;
18         while (mexSet.count(mex)) mex++; // 找到最小的非负整数不在集合中
19         return sg[x] = mex;
20     };
21     for (int i = 0; i <= maxH; i++) if (sg[i] == -1) calcSG(i);
22     int q; cin >> q; // q: 查询次数
23     while (q--) {
```

```

24         int x; cin >> x; // x: 查询的单堆石子数
25         cout << (sg[x] ? "Win" : "Lose") << " ";
26     }
27     cout << "\n";
28     return 0;
29 }

```

3.9 中国剩余定理 (CRT)

解决问题：求解一元线性同余方程组 $x \equiv a_i \pmod{m_i}$ ，其中模数 m_i 两两互质。适用于孙子问题、同余方程求解、RSA-CRT 加速等场景。

题目描述

求解 $x \equiv 1 \pmod{3}$, $x \equiv 2 \pmod{5}$, $x \equiv 3 \pmod{7}$ 。

输入示例

```

3
1 3
2 5
3 7

```

输出示例

52

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4  ll exgcd(ll a, ll b, ll& x, ll& y) {
5      if (!b) { x = 1; y = 0; return a; }
6      ll d = exgcd(b, a % b, y, x);
7      y -= a / b * x;
8      return d;
9  }
10 int main() {
11     int n; // n: 同余方程个数
12     cin >> n;
13     vector<ll> a(n), m(n); // a[i]: 余数, m[i]: 模数 (两两互质)
14     ll M = 1; // M: 所有模数的乘积

```

```

15     for (int i = 0; i < n; i++) { cin >> a[i] >> m[i]; M *= m[i]; }
16     ll ans = 0; // ans: 最终解
17     // 核心原理:  $x = \sum(a_i * M_i * \text{inv}(M_i)) \bmod M$ , 其中  $M_i = M / m_i$ 
18     for (int i = 0; i < n; i++) {
19         ll Mi = M / m[i]; // Mi: M除以m_i
20         ll x, y;
21         exgcd(Mi, m[i], x, y); // 求Mi模m[i]的逆元
22         ans = (ans + a[i] * Mi % M * (x % m[i] + m[i]) % M) % M;
23     }
24     cout << ans << "\n";
25     return 0;
26 }

```

4 算法总结

类型	核心思想	时间复杂度	典型例题
基础数论	试除法/筛法	$O(\sqrt{n})$ 或 $O(n \log \log n)$	质数判断、质因数分解
GCD/扩展欧几里得	辗转相除	$O(\log \min(a, b))$	最大公约数、逆元、同余方程
快速幂/矩阵快速幂	二进制拆分	$O(\log n)$	幂取模、斐波那契、递推加速
组合数学	阶乘逆元、卢卡斯	预处理 $O(n)$, 查询 $O(1)$	组合数、排列计数
高斯消元	初等行变换	$O(n^3)$	线性方程组、异或方程
容斥原理	子集枚举	$O(2^n)$	整除统计、错排
博弈论 SG	SG 定理、mex	状态依赖	Nim 游戏、组合博弈

解题步骤

1. 识别问题类型，确定属于哪个数学领域
2. 分析数据范围，选择合适复杂度的算法
3. 注意取模运算中的溢出和负数处理
4. 对于博弈问题，分析是否为公平组合游戏，尝试递推 SG 值
5. 检验边界条件和特殊情况的正确性

所有代码均已测试，可直接复制运行。